

Universidad de Costa Rica

Sede de Occidente - Recinto de Grecia

Curso: Estructuras de Datos

Proyecto Programado

Sistema de gestión bibliotecaria

Integrantes: Marcos Ferreto Estrada — C5F092

Paulo Anchía Correás — C5C482

Profesora: Iyubanit Rodríguez Ramírez

Fecha: 26 de junio del 2026

Índice

1	Introducción	2
2	Descripción del problema	2
3	Análisis del problema	2
3.1	Estructuras de datos utilizadas	2
3.1.1	Árbol AVL para libros	2
3.1.2	Tabla hash para estudiantes	3
3.1.3	Árbol rojinegro para préstamos	3
3.2	Carga y persistencia de archivos	4
4	Resolución de subproblemas	4
4.1	Funciones o métodos principales	5
5	Análisis de resultados	7
6	Conclusiones y recomendaciones	8
7	Referencias	9

1 Introducción

Este documento describe casi todo el desarrollo del proyecto programado para administrar los datos de la biblioteca de la Universidad de Costa Rica, Recinto de Grecia. El sistema permite cargar, almacenar, consultar, eliminar y prestar libros mediante el uso de estructuras de datos que fueron estudiadas en el curso.

2 Descripción del problema

El problema central consiste en diseñar e implementar una aplicación eficiente que gestione información de libros, estudiantes y préstamos de una biblioteca universitaria utilizando estructuras de datos dinámicas. A diferencia de un sistema trivial basado en arreglos, el volumen de datos de una biblioteca requiere algoritmos capaces de garantizar inserciones, búsquedas y eliminaciones veloces, evitando cuellos de botella cuando el catálogo de libros o el padrón de estudiantes crece de forma considerable.

El sistema debe operar de forma autónoma almacenando todos los datos en archivos de texto plano delimitados (con extensión `.xml`). Al ejecutar el programa, estos datos se cargan en las estructuras en memoria principal: un Árbol AVL para los libros (identificados por un código único), una Tabla Hash para los estudiantes (identificados por su carné), y un Árbol Rojinegro para los préstamos (con códigos autogenerados). La solución exige, además, validaciones de negocio estrictas que mantengan la integridad referencial; por ejemplo, el sistema debe impedir de forma proactiva que se preste un libro que ya se encuentra prestado, o que se elimine el registro de un estudiante que mantenga devoluciones pendientes. Finalmente, todo este procesamiento interno se expone al usuario mediante una Interfaz Gráfica (GUI) construida con `tkinter`, facilitando la interacción diaria del personal bibliotecario.

3 Análisis del problema

3.1 Estructuras de datos utilizadas

3.1.1 Árbol AVL para libros

Los libros se almacenan en un árbol AVL ordenado por su código numérico único. Este método resulta mejor debido a que mantiene un balance automáticamente tras cada inserción o eliminación, asegurando un tiempo de complejidad de $O(\log n)$ en búsquedas, lo cual es importante cuando el catálogo de libros crece.

Datos almacenados por libro:

- Código único.

- Autor.
- Título.
- Año de publicación.
- Editorial.
- Área o temática.

3.1.2 Tabla hash para estudiantes

Los estudiantes se gestionan a través de una tabla hash. La función de dispersión (hash) calcula el índice a partir del número de carné del estudiante utilizando el método de división sobre el tamaño de la tabla, el cual está definido de manera estática en 100 posiciones. Por ejemplo, si se inserta un estudiante con el carné 1001, la función calculará $1001 \pmod{100} = 1$, ubicándolo directamente en el índice 1 del arreglo principal.

Las colisiones (es decir, cuando múltiples carnés producen el mismo índice) se resuelven utilizando el método de encadenamiento a través de listas simples enlazadas. Esta estructura reduce el tiempo de acceso para ubicar el perfil de un estudiante a $O(1)$ en promedio, siendo muy superior a una búsqueda lineal estándar.

Datos almacenados por estudiante:

- Carné único.
- Nombre.
- Carrera que cursa.
- Teléfono.
- Correo electrónico.
- Dirección física.

3.1.3 Árbol rojinegro para préstamos

Los registros de préstamos de libros se indexan en un Árbol Rojinegro. Al tratarse de operaciones transaccionales donde las inserciones y las consultas son frecuentes, esta estructura maneja muy bien el balance sin generar rotaciones excesivas (a diferencia del AVL). Mantiene los tiempos de operación acotados en $O(\log n)$ sin degradarse ante la inserción de fechas u órdenes consecutivos.

Datos almacenados por préstamo:

- Código único de préstamo.
- Código del libro prestado.
- Carné del estudiante solicitante.
- Fecha de préstamo.
- Fecha de devolución esperada.

3.2 Carga y persistencia de archivos

El programa cuenta con un módulo **XMLManager** especializado en la lectura de los archivos de texto **libros.xml**, **estudiantes.xml** y **prestamos.xml**, que pese a su nombre, almacenan atributos delimitados por el carácter de porcentaje (%). Al iniciar la aplicación, se abren estos archivos y cada línea se separa para instanciar directamente los objetos **Libro**, **Estudiante** y **Prestamo**. Posteriormente, al momento de registrar un nuevo préstamo o al ejecutar una eliminación de un estudiante o libro válido, el programa recorre la estructura de datos correspondiente (haciendo uso del recorrido inorden o listando el hash) y sobrescribe los archivos, asegurando la persistencia de los cambios.

4 Resolución de subproblemas

El desarrollo del sistema se abordó de forma incremental, dividiendo el problema general en distintos subproblemas lógicos. Cada subproblema fue resuelto mediante una estructura de datos específica y validaciones de negocio acordes con los requisitos del sistema, siguiendo una implementación lógica y progresiva.

En la siguiente tabla se detalla cómo se abordó la implementación de cada requerimiento.

Subproblema	Solución implementada
Estructura base	Se configuró el proyecto inicial, creando los directorios principales del proyecto y estableciendo las bases para los distintos módulos de código.
Modelos de datos	Se programaron las clases Libro , Estudiante y Prestamo con sus respectivos atributos para representar la información de manera estructurada en el sistema.
Persistencia XML	Se implementó un módulo para leer y escribir en archivos de texto, permitiendo cargar los datos al iniciar y guardar los cambios al finalizar una operación.
Buscar libros por autor	Se realiza un recorrido inorden sobre el árbol AVL, evaluando si el autor del nodo actual coincide o contiene la subcadena ingresada. Cuando existe coincidencia, el libro se agrega a una lista de resultados.
Buscar libro por título	Se aplica un procedimiento similar al de búsqueda por autor, recorriendo el AVL y comparando el título de cada nodo sin distinguir mayúsculas ni minúsculas.
Buscar libro por código	Se aprovecha la propiedad de ordenamiento del AVL. La búsqueda compara recursivamente el código consultado con el código del nodo actual, reduciendo el tiempo de acceso a $O(\log n)$.

Mostrar árboles en inorden	Se implementaron recorridos recursivos que visitan el hijo izquierdo, procesan la raíz y luego el hijo derecho, permitiendo mostrar los elementos en orden ascendente por código.
Buscar estudiante por carné	Se calcula la función hash del carné para obtener el índice correspondiente en la tabla. Luego, se recorre la lista enlazada almacenada en esa posición hasta encontrar el registro exacto.
Buscar estudiante por nombre	Se recorren todas las listas enlazadas de la tabla hash para encontrar coincidencias totales o parciales con el nombre ingresado.
Buscar estudiantes por carrera	Se sigue un procedimiento similar al de búsqueda por nombre, recorriendo toda la tabla y acumulando los registros que coinciden con la carrera consultada.
Estructura Rojinegra	Se implementó el árbol rojinegro con sus respectivas rotaciones e inserciones balanceadas para almacenar de manera eficiente los préstamos registrados.
Sistema de préstamos	Primero se verifica la existencia del libro y del estudiante. Luego se comprueba que el libro no esté prestado. Si todo es correcto, se crea el préstamo con duración de 15 días, se inserta en el árbol rojinegro y se actualiza el archivo.
Eliminar libro por código	Antes de eliminar, se valida en el árbol rojinegro que el libro no esté asociado a un préstamo activo. Si el libro está prestado, la eliminación se rechaza. En caso contrario, se elimina del AVL y se rebalancea el árbol.
Eliminar estudiante por carné	Antes de eliminar, el sistema verifica que el estudiante no tenga préstamos activos en el árbol rojinegro. Si existe al menos uno, la eliminación se bloquea. Si no, se elimina el nodo correspondiente de la tabla hash.
Interfaz gráfica	Se desarrolló con <code>tkinter</code> , organizando la aplicación en pestañas mediante <code>Notebook</code> para separar libros, estudiantes y préstamos. Además, se incorporaron formularios de entrada, botones de acción y un <code>Treeview</code> tabular para mostrar resultados.
Documentación interna	Se agregaron <i>docstrings</i> y comentarios descriptivos en el código, detallando los parámetros, retornos y el funcionamiento algorítmico de cada función.

4.1 Funciones o métodos principales

Las funciones y métodos principales del sistema se diseñaron para cubrir las operaciones esenciales de carga, búsqueda, eliminación, préstamo y guardado de información. Cada uno cumple una función específica dentro del flujo general del programa y permite separar la lógica de negocio de la interfaz gráfica.

Función o método	Descripción
<code>cargar_todo()</code>	Parámetros: Ninguno. Retorno: tuple con listas de objetos. Hace: Método central de <code>XMLManager</code> que lee los tres archivos .xml y devuelve los registros instanciados en memoria.
<code>cargar_[entidad]()</code>	Parámetros: Ninguno. Retorno: List. Hace: Métodos específicos de <code>XMLManager</code> que parsean y retornan los objetos individuales según el formato de cada archivo.
<code>guardar_[entidad]()</code> <code>_guardar_[entidad]_xml()</code>	/ Parámetros: List de objetos (en la variante pública). Retorno: Ninguno. Hace: Operan en dos capas: los métodos privados <code>_guardar*_xml()</code> de <code>GestorEliminacion</code> y <code>SistemaPrestamos</code> recolectan los datos de las estructuras en memoria, y luego delegan la escritura a los métodos públicos <code>guardar*()</code> de <code>XMLManager</code> , que sobrescriben el archivo en disco.
<code>insertar()</code>	Parámetros: El objeto a insertar (Libro, Estudiante, etc.). Retorno: Nuevo nodo raíz. Hace: Agrega un nuevo registro y rebalancea el árbol si es necesario.
<code>buscar_codigo()</code>	Parámetros: <code>codigo: int</code> . Retorno: Objeto encontrado o None. Hace: Localiza rápidamente un libro o préstamo aprovechando el orden del AVL/Rojinegro.
<code>buscar_autor()</code>	Parámetros: <code>autor: str</code> . Retorno: list[Libro]. Hace: Recorre inorden el AVL y acumula todos los libros cuyo autor coincida con la cadena recibida.
<code>buscar_titulo()</code>	Parámetros: <code>titulo: str</code> . Retorno: Optional[Libro]. Hace: Recorre inorden el AVL y devuelve el primer libro cuyo título coincida exactamente.
<code>buscar_por_carnet()</code>	Parámetros: <code>carnet: int</code> . Retorno: Estudiante o None. Hace: Accede a la tabla hash mediante el módulo y extrae el estudiante exacto.
<code>buscar_por_nombre()</code>	Parámetros: <code>nombre: str</code> . Retorno: Lista de coincidencias. Hace: Recorre las listas enlazadas del hash buscando coincidencias de atributos.
<code>libro_esta_prestado()</code>	Parámetros: <code>codigo: int</code> . Retorno: bool. Hace: Verifica en el árbol rojinegro si existe un préstamo activo para evitar redundancias o borrados.
<code>estudiante_tiene_prestamos()</code>	Parámetros: <code>carnet: int</code> . Retorno: bool. Hace: Valida que el estudiante no tenga devoluciones pendientes antes de proceder a una eliminación.
<code>dar_prestamo()</code>	Parámetros: <code>cod_libro, carnet</code> . Retorno: Tuple[bool, str]. Hace: Comprueba existencia y disponibilidad, e inserta el nuevo registro en el Rojinegro.

<code>eliminar_libro()</code> <code>eliminar_estudiante()</code>	/	Parámetros: <code>codigo/carnet: int.</code> Retorno: <code>Tuple[bool, str].</code> Hace: Controladores en <code>GestorEliminacion</code> que ejecutan un borrado seguro de la estructura si las validaciones pasan con éxito.
<code>obtener_libros_inorden()</code> <code>inorden()</code>	/	Parámetros: Nodo raíz. Retorno: <code>List.</code> Hace: Recorrido inorden (izquierdo $\rightarrow$ raíz $\rightarrow$ derecho) que devuelve los elementos ordenados por código. <code>ArbolAVL</code> expone esta operación como <code>obtener_libros_inorden()</code> ; <code>ArbolRojinegro</code> como <code>inorden()</code> . Mismo patrón, distinto nombre por clase.

5 Análisis de resultados

La siguiente tabla resume el grado de completitud técnica de las diferentes etapas y requerimientos del sistema en el momento de la entrega del código final.

Etapas	Requerimiento	Estado		Observaciones
Etapas 1	Estructura base	Terminado éxito	con	Directorios y módulos iniciales configurados.
Etapas 2	Modelos de datos	Terminado éxito	con	Clases implementadas con sus atributos.
Etapas 3	Persistencia XML	Terminado éxito	con	Uso correcto de la persistencia directa (%).
Etapas 4	Árbol AVL y Búsquedas	Terminado éxito	con	Funcional con rebalanceo, inserción y búsqueda.
Etapas 5	Tabla Hash	Terminado éxito	con	Funcional con resolución de colisiones por listas.
Etapas 6	Árbol Rojinegro	Terminado éxito	con	Funcional en balanceo por colores para préstamos.
Etapas 7	Sistema de préstamos	Terminado éxito	con	Validación lógica estricta y control de fechas.
Etapas 8	Eliminaciones seguras	Terminado éxito	con	Validaciones integradas que impiden corromper.
Etapas 9	Interfaz gráfica (GUI)	Terminado éxito	con	Pantallas organizadas por pestañas, vistas limpias.
Etapas 10	Documentación interna	Terminado éxito	con	Funciones documentadas e informe elaborado.

6 Conclusiones y recomendaciones

1. **Conclusión:** El uso adecuado de las estructuras de datos mejoró sustancialmente la eficiencia y escalabilidad del sistema comparado a utilizar simples arreglos secuenciales.
Recomendación: Es recomendable mantener un análisis de complejidad inicial antes de empezar futuros desarrollos para escoger el modelo de datos correcto desde la fase de planeación.
2. **Conclusión:** El Árbol AVL demostró ser altamente útil y óptimo para mantener los códigos de libros ordenados y responder rápidamente a búsquedas exactas.
Recomendación: En implementaciones donde las eliminaciones de libros sean abrumadoramente masivas, el re-balanceo constante podría penalizar el rendimiento, por lo que podría recomendarse estudiar alternativas de baja frecuencia en ese caso hipotético.
3. **Conclusión:** La tabla hash permitió el acceso directo y ágil al registro de los estudiantes mediante su número de carné, optimizando significativamente la operación.
Recomendación: Para bases de datos universitarias reales masivas, se recomienda calibrar y testear el tamaño de la tabla (actualmente 100) frente al uso esperado para no castigar el factor de carga ni generar listas de colisión excesivamente largas.
4. **Conclusión:** El manejo del Árbol Rojinegro brindó la estabilidad necesaria para las operaciones de préstamos, las cuales usualmente implican un crecimiento secuencial del identificador.
Recomendación: Aprovechar el ordenamiento del árbol rojinegro para en una futura iteración añadir reportes automáticos de libros próximos a vencer su fecha de devolución.
5. **Conclusión:** La programación de las validaciones de dependencia previas (comprobar si hay un préstamo antes de permitir borrar un libro) previno proactivamente las inconsistencias en disco e interrupciones del programa.
Recomendación: Mantener una lógica fuerte del lado del controlador de persistencia antes de alterar memoria, para que sirva de segunda capa de validación.
6. **Conclusión:** La Interfaz Gráfica de Usuario (GUI) construida con los componentes nativos, le dio un aspecto intuitivo y organizado a una lógica subyacente que de otra forma sería compleja de probar por comandos.
Recomendación: Se puede ampliar el sistema incluyendo filtros avanzados por fechas, reportes PDF o alertas visuales cuando un estudiante acumula penalizaciones y morosidades, lo cual complementaría excelentemente a la GUI construida.
7. **Conclusión:** El manejo de notificaciones se implementó de forma muy robusta mediante una barra de estado en el pie de página (footer) de la interfaz. Esto asegura que se muestre un error explícito si faltó algún dato, o un mensaje de éxito confirmando exactamente qué registro se insertó, buscó o eliminó al interactuar con el sistema.
Recomendación: Conservar y escalar este patrón de diseño en futuros módulos de la aplicación, ya que ofrece una excelente retroalimentación en tiempo real al usuario sin interrumpir su flujo de trabajo con ventanas invasivas.
8. **Conclusión:** El abordaje del desarrollo mediante una división incremental por etapas facilitó la detección temprana de errores y permitió probar la integración de cada estructura de forma aislada antes del acoplamiento final en la GUI.

Recomendación: Adoptar metodologías ágiles o herramientas de control de tareas en futuros proyectos grupales o de mayor escala, con el fin de mejorar la trazabilidad de los requerimientos y mantener un código ordenado.

7 Referencias

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.

GeeksforGeeks. (2024). *Red-Black Tree Data Structure*. <https://www.geeksforgeeks.org/red-black-tree-set-1-introduction-2/>

Python Software Foundation. (2024). *tkinter — Python interface to Tcl/Tk*. <https://docs.python.org/es/3/library/>

Rodríguez Ramírez, I. (2026). *Estructuras de datos* [Diapositivas de curso]. Universidad de Costa Rica, Recinto de Grecia.

Shipman, J. W. (2013). *Tkinter 8.5 reference: A GUI for Python*. New Mexico Tech Computer Center. <https://www.nmt.edu/tcc/help/pubs/tkinter/web/index.html>